GoogLeNet 前哨站

訓練須知

目錄

- AlexNet 講解
- Overfitting
- 核與計算量

Part 1.

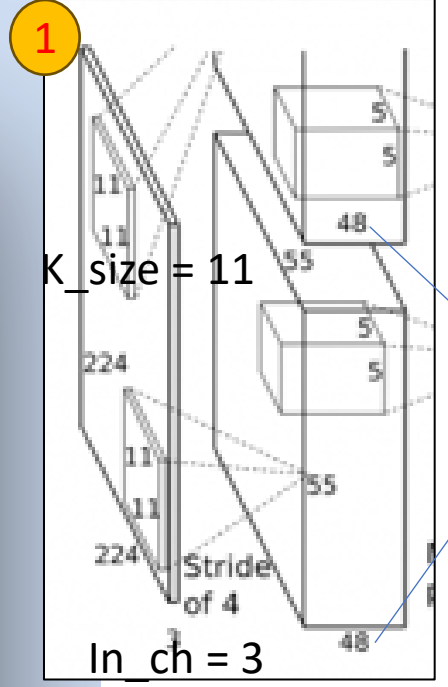
AlexNet

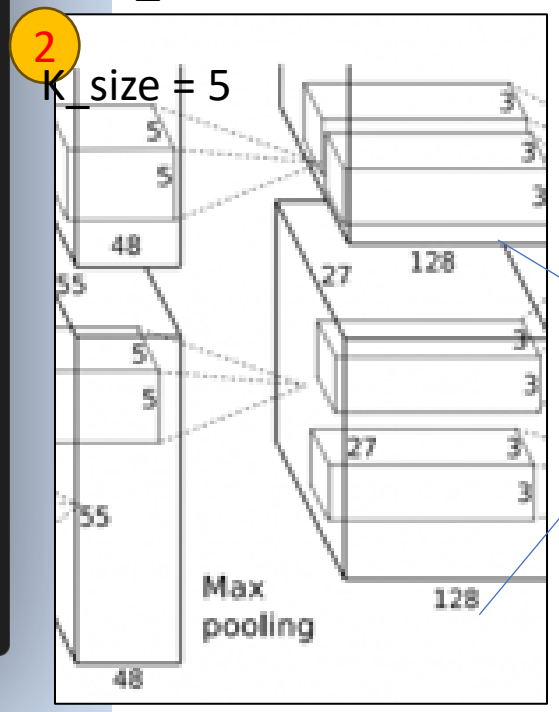
```python
#為了方便模組化，會將模型拆成 1.features(卷積) 2.classifier(分類用的全連接層)
class AlexNet(nn.Module):
    def __init__(self, in_channels=3, num_classes=1000):
        super(AlexNet, self).__init__()
        #1. features
        self.features = nn.Sequential(
            #conv1:
            nn.Conv2d(in_channels, 96, kernel_size=11, stride=4, padding=2), #224x224 -> 55x55
            nn.ReLU(inplace=True),#激活函數：擬合非線性複雜資料 f(x) = max(0, x)
            nn.MaxPool2d(kernel_size=3, stride=2), #55x55 -> 27x27

            #conv2(same padding):
            nn.Conv2d(96, 256, kernel_size=5, padding=2), #27x27 -> 27x27
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2), #27x27 -> 13x13
            #conv3(same padding), 4(same padding), 5:
            nn.Conv2d(256, 384, kernel_size=3, padding=1), #13x13 -> 13x13
            nn.ReLU(inplace=True),
            nn.Conv2d(384, 384, kernel_size=3, padding=1), #13x13 -> 13x13
            nn.ReLU(inplace=True),
            nn.Conv2d(384, 256, kernel_size=3, padding=1), #13x13 -> 13x13
            nn.ReLU(inplace=Ture),
            nn.MaxPool2d(kernel_size=3, stride=2), # #13x13 -> 6x6
        )
```

1

2

3



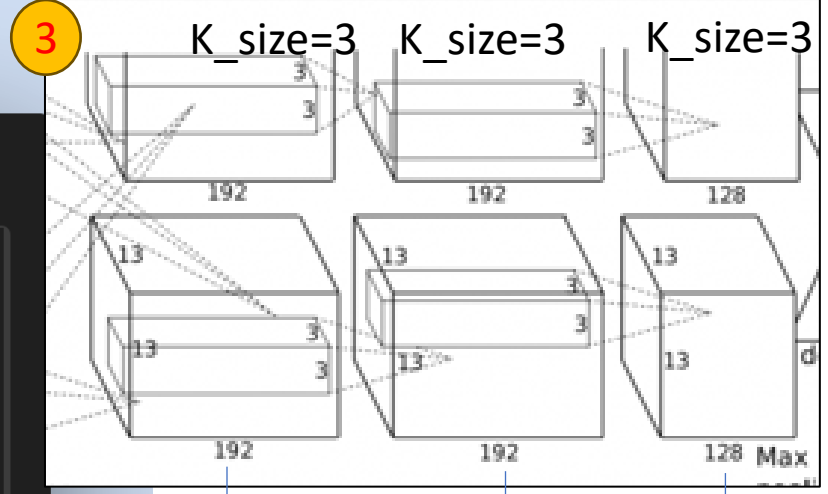


```python
#為了方便模組化，會將模型拆成 1.features(卷積) 2.classifier(分類用的全連接層)
class AlexNet(nn.Module):
    def __init__(self, in_channels=3, num_classes=1000):
        super(AlexNet, self).__init__()
        #1. features
        self.features = nn.Sequential(
            #conv1:
            nn.Conv2d(in_channels, 96, kernel_size=11, stride=4, padding=2), #224x224 -> 55x55
            nn.ReLU(inplace=True),#激活函數：擬合非線性複雜資料 f(x) = max(0, x)
            nn.MaxPool2d(kernel_size=3, stride=2), #55x55 -> 27x27

            #conv2(same padding):
            nn.Conv2d(96, 256, kernel_size=5, padding=2), #27x27 -> 27x27
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2), #27x27 -> 13x13
            #conv3(same padding), 4(same padding), 5:
            nn.Conv2d(256, 384, kernel_size=3, padding=1), #13x13 -> 13x13
            nn.ReLU(inplace=True),
            nn.Conv2d(384, 384, kernel_size=3, padding=1), #13x13 -> 13x13
            nn.ReLU(inplace=True),
            nn.Conv2d(384, 256, kernel_size=3, padding=1), #13x13 -> 13x13
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2), # #13x13 -> 6x6
        )
```

1

2

3

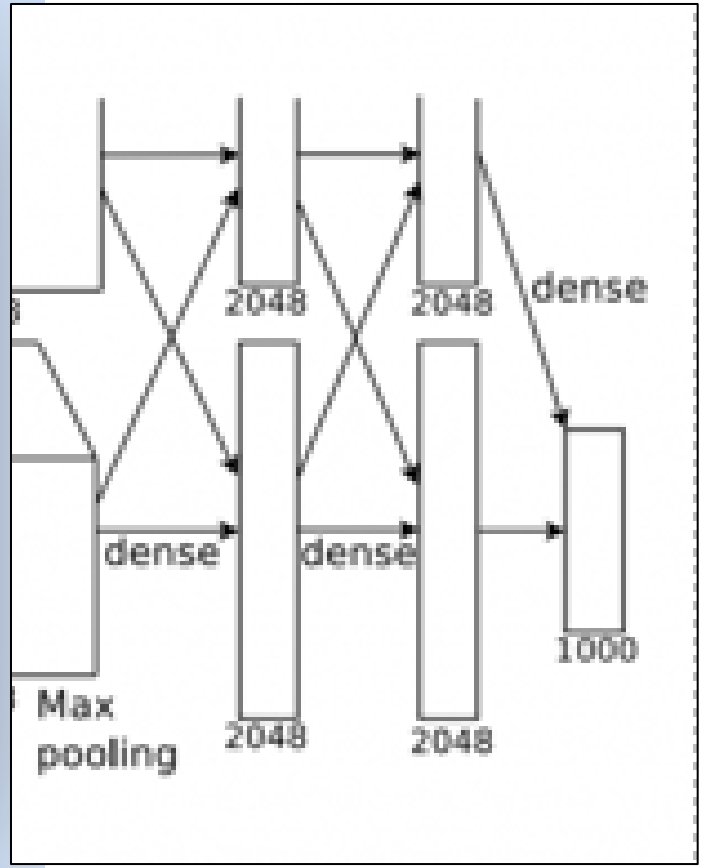


Ch=384 Ch=384 Ch=256

```python
        self.classifier = nn.Sequential(
            #nn.Dropout(), #prevent overfitting : 關閉特定神經元，讓剩下的學會提取特徵
            nn.Linear(256 * 6 * 6, 4096),
            nn.ReLU(inplace=True),
            #nn.Dropout(),
            nn.Linear(4096, 4096),
            nn.ReLU(inplace=True),
            nn.Linear(4096, num_classes),
        )

    def forward(self, x):
        x = self.features(x)
        x = torch.flatten(x, 1)
        x = self.classifier(x)
        return(x)
```

MaxPooling後13->6

壓成1D，送到FC

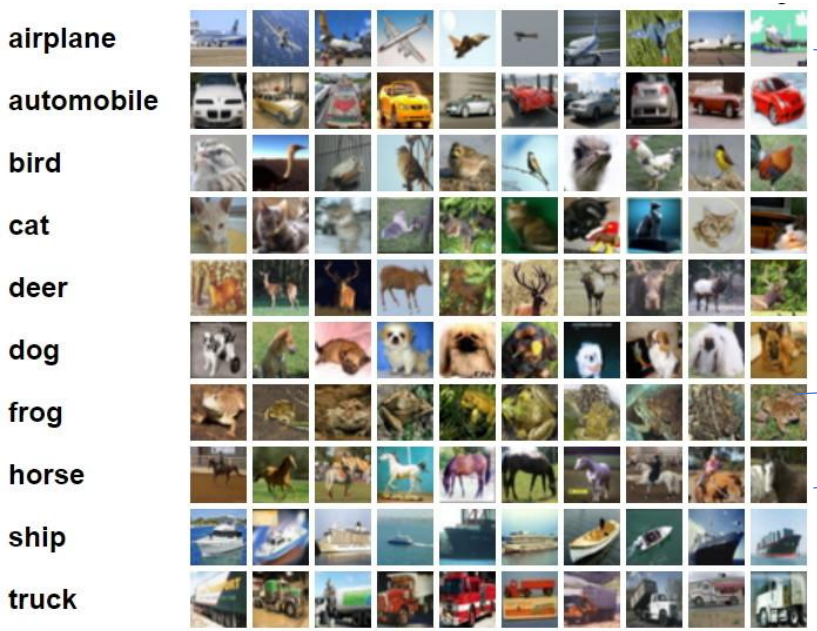


Part 2.

overfitting

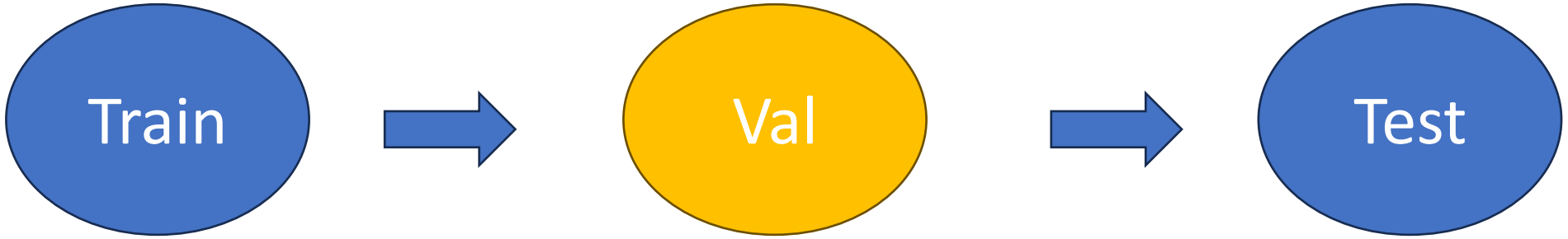
Generalization

機器學習的目標就是要訓練機器擁有人類的思考，並且擁有解決一般問題的能力，即使看到沒有包含在訓練資料的資料，也是要可以正確辨識的。



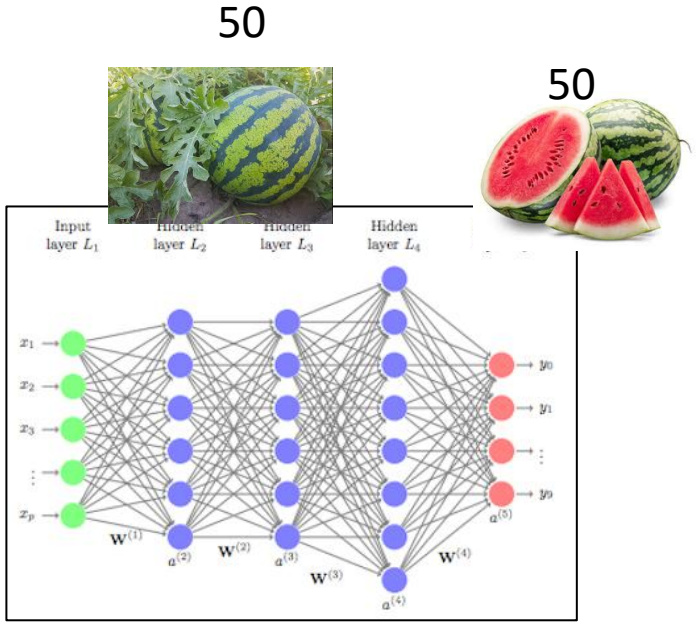
Train vs. Test Split

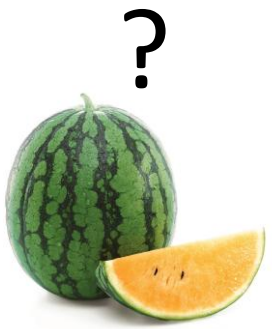
- 我們會將數據集分成兩份，一份用來讀書，一份用來考試
- 如果訓練分數很高，測試分數很低，這可能代表overfitting



Overfitting 狀況

- **樣本太少，模型太大**





- 訓練次數過多

- 數據雜訊過多

Overfitting 狀況 – 小實驗

- 載入cifar 10資料集

CIFAR-10 是一個經典的計算機視覺資料集，包含 60,000 張 32 × 32 像素的彩色影像，分為 10 個類別（飛機、汽車、鳥、貓、鹿、狗、青蛙、馬、船、卡車）。它由 50,000 張訓練影像和 10,000 張測試影像組成，常

```python
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

# 定義資料轉換流程
transform = transforms.Compose([
    #transforms.Resize(224),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])

train_set = datasets.CIFAR10(root='./data', train=True, download=True, transform=transform)
test_set = datasets.CIFAR10(root='./data', train=False, download=True, transform=transform)
```

✓ 1.4s

Overfitting 狀況 – 小實驗

- 製作50張的子集

```
from torch.utils.data import Subset

# 製作子資料集
train_subset = Subset(train_set, range(50))
test_subset = Subset(test_set, range(50))

train_loader = DataLoader(train_set, batch_size=10, shuffle=True)
test_loader = DataLoader(test_set, batch_size=10, shuffle=False)
print(f"訓練子集數量: {len(train_subset)}")
```

** 先修改一下AlexNet **

改輸入層捲積核

```
#1. features
self.features = nn.Sequential(
    #conv1:
    nn.Conv2d(in_channels, 96, kernel_size=3, stride=1, padding=1), #224x224 -> 55x55
    nn.ReLU(inplace=True),#激活函數：擬合非線性複雜資料 f(x) = max(0, x)
    nn.MaxPool2d(kernel_size=3, stride=2), #55x55 -> 27x27
```

** 先修改一下AlexNet **

```
    nn.MaxPool2d(kernel_size=3, stride=2), # #13x13 -> 6x6
)
self.avgpool = nn.AdaptiveAvgPool2d((3, 3))
self.classifier = nn.Sequential(
    #nn.Dropout(p=0.5), #prevent overfitting : 關閉特定神經元，讓剩下的學會提取特徵
    #nn.Linear(256 * 6 * 6, 4096),
    nn.Linear(256 * 3 * 3, 1024),
    nn.ReLU(inplace=True),
    #nn.Dropout(p=0.5),
    nn.Linear(1024, 512),
    #nn.Linear(4096, 4096),
    nn.ReLU(inplace=True),
    #nn.Linear(4096, num_classes),
    nn.Linear(512, num_classes),
)
```

加入Adaptive Pool, 確保捲積完是3x3

全連接層修改成相應尺寸 -> 1024

最後是512 -> 10 (512比較適合cifar10)

Overfitting 狀況 – 小實驗

```
import matplotlib.pyplot as plt
import torch
import torch.optim as optim
from torchvision import models
import torch.nn as nn
from tqdm import tqdm
```

```
train_acc_his = []
test_acc_his = []

model = AlexNet(num_classes=10).cuda()
optimizer = optim.Adam(model.parameters(), lr=0.001)
criterion = nn.CrossEntropyLoss()
```

紀錄準確率

Cifar 10的固定類別

Overfitting 狀況 – 小實驗

```python
# 2. 訓練循環 (觀察 Loss 和 Acc)
for epoch in range(40):
    model.train()
    train_loss, train_correct = 0, 0
    for images, labels in tqdm(train_loader):
        images, labels = images.cuda(), labels.cuda()

        outputs = model(images)
        loss = criterion(outputs, labels)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        train_loss += loss.item()
        _, predicted = outputs.max(1)
        train_correct += predicted.eq(labels).sum().item()
```

Overfitting 狀況 – 小實驗

```
#
model.eval()
test_correct = 0
with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.cuda(), labels.cuda()
        outputs = model(images)
        _, predicted = outputs.max(1)
        test_correct += predicted.eq(labels).sum().item()
```

```
print(f"Epoch {epoch+1}:Loss: {train_loss/len(train_loader):.4f}
# ... 在 Epoch 結束後 ...
train_acc_his.append(100.*train_correct/len(train_subset))
test_acc_his.append(100.*test_correct/len(test_subset))
```

```
| Train Acc: {100.*train_correct/len(train_subset):.2f}% | Test Acc: {100.*test_correct/len(test_subset):.2f}%")
```

Overfitting 狀況 – 小實驗

```
# 訓練結束後畫圖
plt.plot(train_acc_his, label='Train Acc')
plt.plot(test_acc_his, label='Test Acc')
plt.xlabel('Epoch')
plt.ylabel('Accuracy (%)')
plt.legend()
plt.title('Overfitting Visualization')
plt.show()
```

作業1 – 實作訓練

- 試看看調整訓練圖片張數 (epoch自調)
- 1.拿掉Dropout層 2.或調整drop比率
- 回到一開始，用Cifar10訓練原本的AlexNet (原有模型架構改回去，輸入要resize 224，把註解拿掉就可以跑了)

請觀察train與test acc、loss變化，說明在進行上述的任務時，你的發現。(請使用簡報)

Part 2.

核與計算量

1.傳統方式 k=5 (假定 64-> 128)

FLOPs = 輸出H × 輸出W × 輸出channel × 輸入channel × kernel² × 2

↑

乘法+加法算2次

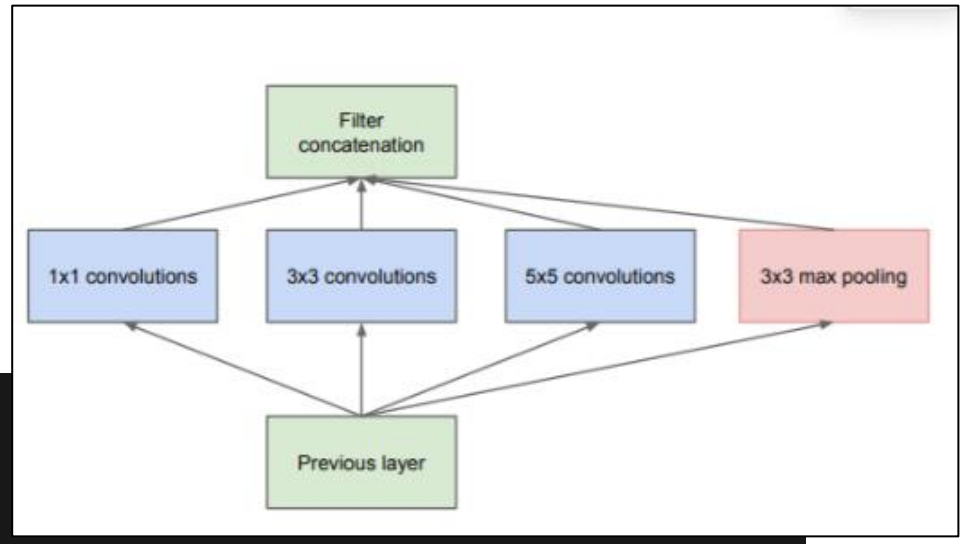
```python
class NormalBlock(nn.Module):
    def __init__(self, in_channel, out_channel):
        super().__init__()
        self.conv = nn.Conv2d(in_channel, out_channel, kernel_size=5, padding=2)

    def forward(self, x):
        return self.conv(x)
```

✓ 0.0s

假設same padding:

FLOPs = 28x28x128x64x5x5x2 = **160M**

2.Inception Block



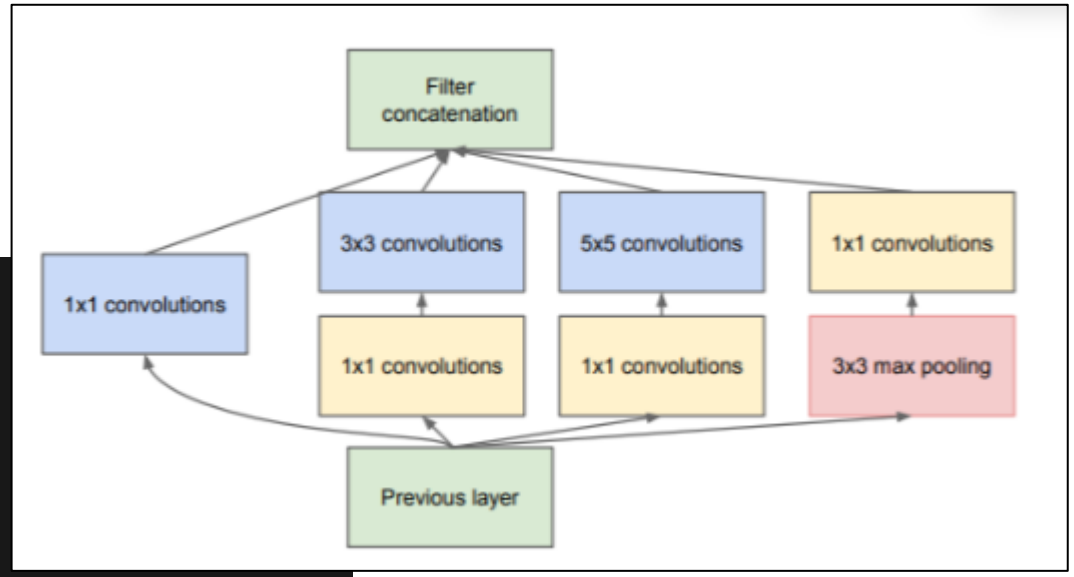

```python
class InceptionOriginal(nn.Module):
    def __init__(self, inchannel):
        super().__init__()
        self.b1 = nn.Conv2d(inchannel, 32, kernel_size=1)
        self.b2 = nn.Conv2d(inchannel, 32, kernel_size=3, padding=1)
        self.b3 = nn.Conv2d(inchannel, 32, kernel_size=5, padding=2)
        self.b4 = nn.MaxPool2d(3, stride=1, padding=1)
        self.b4_conv = nn.Conv2d(inchannel, 32, kernel_size=1)

    #串聯網路 (Parallel)
    def forward(self, x):
        return torch.cat([self.b1(x), self.b2(x), self.b3(x), self.b4_conv(self.b4(x))], 1)
```

假設same padding:

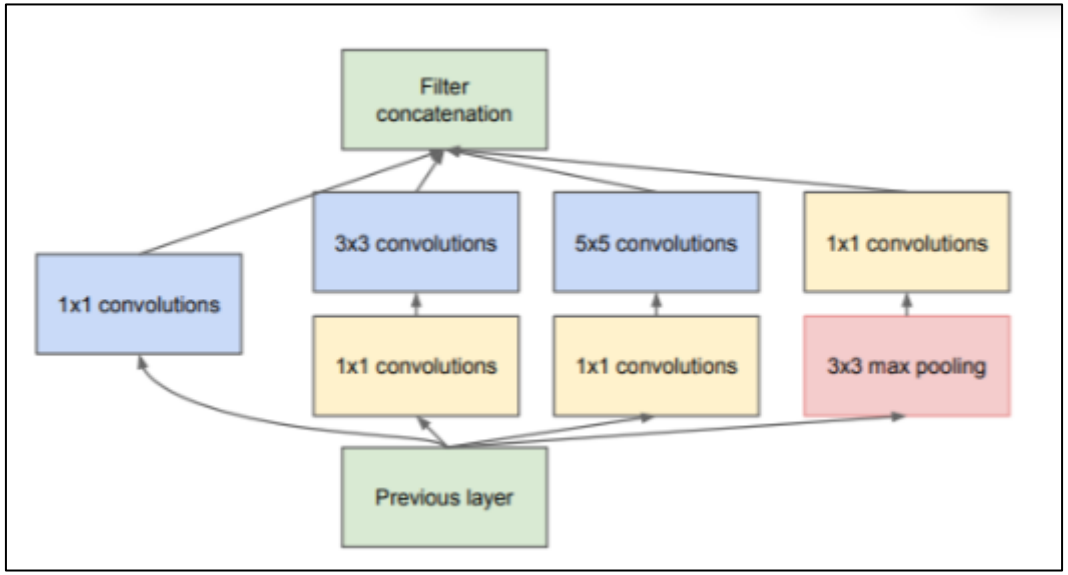
FLOPs = $28 \times 28 \times 32 \times 64 \times 1^2 \times 2 + 28 \times 28 \times 32 \times 64 \times 3^2 \times 2 + 28 \times 28 \times 32 \times 64 \times 5^2 \times 2 + 28 \times 28 \times 32 \times 64 \times 1^2 \times 2$ = **115.6 M**

3.Inception Block – 改

```python
class GoogleInception(nn.Module):
    def __init__(self, in_channel):
        super().__init__()
        self.b1 = nn.Conv2d(in_channel, 32, kernel_size=1)
        self.b2 = nn.Sequential(
            nn.Conv2d(in_channel, 16,kernel_size=1),
            nn.Conv2d(16, 32, kernel_size=3, padding=1)
        )
        self.b3 = nn.Sequential(
            nn.Conv2d(in_channel, 16, kernel_size=1),
            nn.Conv2d(16, 32, kernel_size=5, padding=2)
        )
        self.b4 = nn.Sequential(
            nn.MaxPool2d(3, stride=1, padding=1),
            nn.Conv2d(in_channel, 32, kernel_size=1)
        )

    def forward(self, x):
        return torch.cat([self.b1(x), self.b2(x), self.b3(x),self.b4(x)], 1)
```



3.Inception Block – 改



FLOPs =

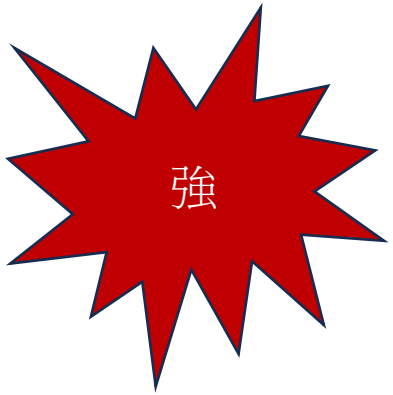
1x1 : $28 \times 28 \times 32 \times 64 \times 1^2 \times 2 = 3.2M$

1x1 : $28 \times 28 \times 16 \times 64 \times 1^2 \times 2 = 1.6$ M
3x3 : $28 \times 28 \times 32 \times 16 \times 3^2 \times 2 = 7.2$ M

1x1 : $28 \times 28 \times 16 \times 64 \times 1^2 \times 2 = 1.6M$
5x5 : $28 \times 28 \times 32 \times 16 \times 5^2 \times 2 = 20.1$ M

Maxpool -> 0
1x1 : $28 \times 28 \times 32 \times 64 \times 1^2 \times 2 = 3.2$ M

結論: 36.9 M



比較看看吧

```python
input_data = torch.randn(1, 64, 28, 28)

models = [
    ("NormalBlock", NormalBlock(64, 128)),
    ("InceptionOriginal", InceptionOriginal(64)),
    ("GoogleInception", GoogleInception(64))
]
print(f"{'架構名稱':<30} | {'參數 (Params)':<15} | {'運算量 (FLOPs)':<15}")
for name, model in models:
    flops, params = profile(model, inputs =(input_data, ),verbose=False)
    print(f"{name:<33} | {params:<17.0f} | {flops/1e6:} M")
```

✓ 0.0s

架構名稱	參數 (Params)	運算量 (FLOPs)
NormalBlock	204928	160.5632 M
InceptionOriginal	73856	57.802752 M
GoogleInception	23712	18.464768 M

GoogLeNet特色與影響

- 2014 年 ImageNet比賽(ILSVRC14)第一名
- 提出了Inception架構(多尺度的卷積處理)
- 把Ensemble的思想用到一個極致
- 提出1x1的卷積
- 據說GoogLeNet的L要大寫是為了致敬LeNet

參考

GoogleNet介紹: https://www.youtube.com/watch?v=agA3CDDIjZw&t=285s

GoogLeNet論文：https://arxiv.org/pdf/1409.4842.pdf

Pytorch建inception-v1: https://www.youtube.com/watch?v=uQc4Fs7yx5I&t=675